

On the criteria to be used in decomposing systems into modules - Artigo de Opinião

O artigo *"On the Criteria To Be Used in Decomposing Systems into Modules"*, de David L. Parnas, nos engana ao se assemelhar com um texto técnico sobre organização de sistemas, mas na prática ele faz uma crítica mais profunda: a forma como desenvolvedores pensam software costuma ser superficial.

Parnas questiona a ideia tradicional de dividir sistemas com base em etapas de processamento, algo que, se olharmos superficialmente, parece lógico. No entanto, essa lógica ignora um fator essencial: sistemas mudam. E com frequência. Quando a estrutura é pensada apenas como um fluxo contínuo, qualquer alteração vira um problema espalhado pelo sistema inteiro.

O que o autor propõe, não é apenas uma técnica diferente, mas uma mudança de mentalidade. Parnas sugere que os módulos devem ser organizados para esconder decisões que podem variar com o tempo. Deslocando o foco do funcionamento atual do sistema para como o sistema pode funcionar depois de melhorado. E é exatamente onde a maioria dos projetos apresenta falhas.

O interessante é que essa ideia parece óbvia nos dias atuais, mas claramente não era na época, e até mesmo hoje, ainda não é aplicada como deveria. É comum ver sistemas que até são divididos em módulos, mas continuam altamente acoplados, justamente porque não escondem nada de relevante. Apresentando uma falsa modularização que só funciona na fachada.

Um outro ponto que chama atenção é que Parnas antecipa conceitos que hoje são básicos na produção de software, como encapsulamento e separação de responsabilidades. Mas ele faz isso de uma forma mais estratégica, não só como uma regra de design, mas como uma forma de lidar com a inevitabilidade da mudança.

Ao mesmo tempo, aplicar esse princípio não é tão simples quanto o artigo pode fazer parecer. Exige prever quais decisões são realmente voláteis, o que tende a não ser tão óbvio. Existe um certo nível de maturidade que, de modo prático, muitos projetos ainda não têm.

Por fim, o artigo deixa uma sensação meio incômoda: a de que muitos dos problemas atuais em software não existem por falta de técnica, mas por insistência em pensar de forma limitada. Parnas não apenas nos propõe uma nova solução, ele está apontando erros de raciocínio que ainda persistem.

E é exatamente por isso que o texto continua relevante, mesmo depois de tantos anos. Não porque ele nos traz uma solução nova ou revolucionária, mas porque expõe algo está presente na atualidade ainda de forma palpável.